



TRIOS CYBER

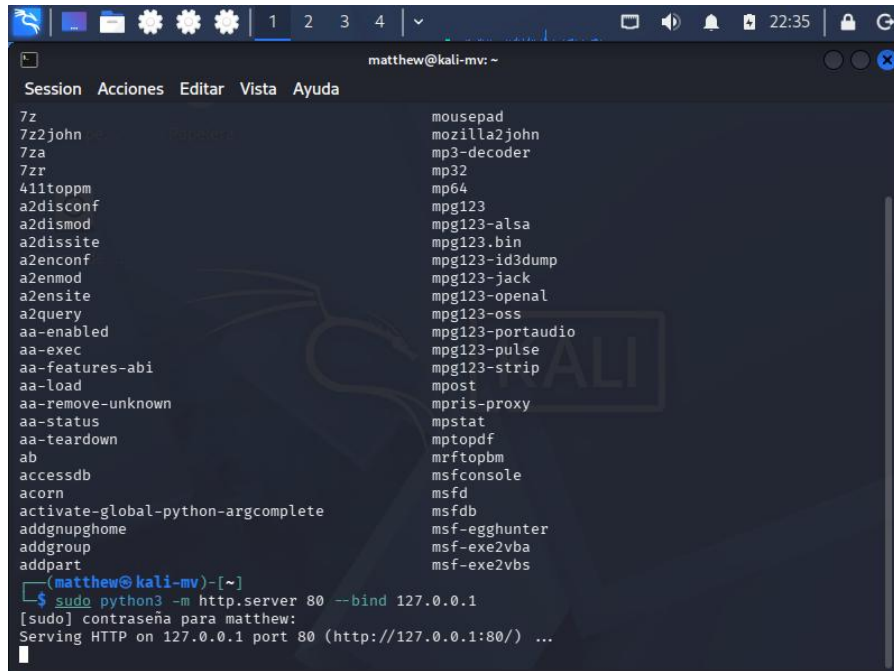
LABORATORY PRACTICE 4

TCP Connection Establishment and Teardown Analysis

Name: Matthew Josue Portilla Morejon

1. Local TCP Service Setup

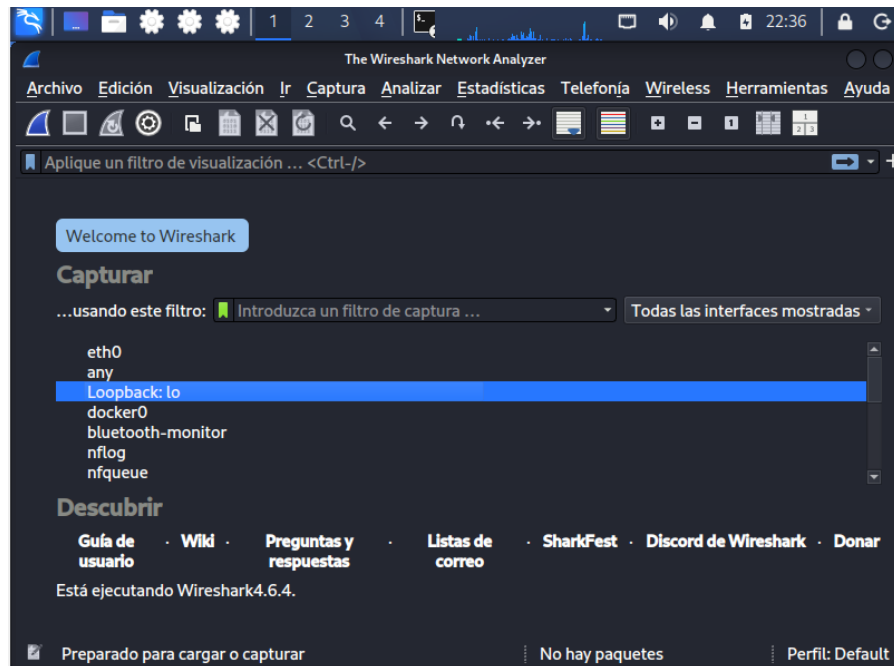
An authorized local HTTP service was started on Kali Linux using Python and bound to the loopback address 127.0.0.1 on TCP port 80. This provided a controlled local service for generating the TCP connection required by the laboratory.



```
matthew@kali-mv: ~  
Session Acciones Editar Vista Ayuda  
7z  
7z2john  
7za  
7zr  
411toppm  
a2disconf  
a2dismod  
a2dissite  
a2enconf  
a2enmod  
a2ensite  
a2query  
aa-enabled  
aa-exec  
aa-features-abi  
aa-load  
aa-remove-unknown  
aa-status  
aa-teardown  
ab  
accessdb  
acorn  
activate-global-python-argcomplete  
addgnupghome  
addgroup  
addpart  
mousepad  
mozilla2john  
mp3-decoder  
mp32  
mp64  
mpg123  
mpg123-alsa  
mpg123.bin  
mpg123-id3dump  
mpg123-jack  
mpg123-openal  
mpg123-oss  
mpg123-portaudio  
mpg123-pulse  
mpg123-strip  
mpost  
mpris-proxy  
mpstat  
mptopdf  
mrftopbm  
msfconsole  
msfd  
msfdb  
msf-egg hunter  
msf-exe2vba  
msf-exe2vbs  
$ sudo python3 -m http.server 80 --bind 127.0.0.1  
[sudo] contraseña para matthew:  
Serving HTTP on 127.0.0.1 port 80 (http://127.0.0.1:80/) ...
```

2. Wireshark Capture Preparation

Wireshark was opened on the Kali Linux virtual machine and the loopback interface (lo) was selected. This interface was used because both the client and the authorized HTTP service communicated locally through 127.0.0.1.



3. TCP Connection Generation

A TCP connection to the local HTTP service was generated with curl using `http://127.0.0.1`. The request successfully reached the local service and returned the HTTP directory response, confirming that a complete TCP session had been created for analysis.

```

matthew@kali-mv: ~
$ curl http://127.0.0.1
<!DOCTYPE HTML>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Directory listing for /</title>
</head>
<body>
<h1>Directory listing for /</h1>
<hr>
<ul>
<li><a href=".bash_logout">.bash_logout</a></li>
<li><a href=".bashrc">.bashrc</a></li>
<li><a href=".bashrc.original">.bashrc.original</a></li>
<li><a href=".cache">.cache</a></li>
<li><a href=".config">.config</a></li>
<li><a href=".dmrc">.dmrc</a></li>
<li><a href=".face">.face</a></li>
<li><a href=".face.icon">.face.icon@</a></li>
<li><a href=".gnupg">.gnupg</a></li>
<li><a href=".ICEauthority">.ICEauthority</a></li>
<li><a href=".java">.java</a></li>
<li><a href=".local">.local</a></li>
<li><a href=".mozilla">.mozilla</a></li>
<li><a href=".msf4">.msf4</a></li>
<li><a href=".profile">.profile</a></li>
<li><a href=".ssh">.ssh</a></li>
<li><a href=".sudo_as_admin_successful">.sudo_as_admin_successful</a></li>
<li><a href=".vboxclient-clipboard-tty7-control.pid">.vboxclient-clipboard-tty7-control.pid</a></li>

```

4. Three-Way Handshake Identification

The captured traffic was filtered with `tcp.port == 80`. The beginning of the session showed the expected TCP three-way handshake: a SYN from client port 58300 to server port 80, a SYN-ACK from port 80 back to 58300, and a final ACK from 58300 to port 80 before the HTTP request was transmitted.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.1	TCP	74	58300 → 80 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TS
2	0.000013172	127.0.0.1	127.0.0.1	TCP	74	80 → 58300 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 S
3	0.000022029	127.0.0.1	127.0.0.1	TCP	66	58300 → 80 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=3867338285
4	0.000059952	127.0.0.1	127.0.0.1	HTTP	139	GET / HTTP/1.1
5	0.000063639	127.0.0.1	127.0.0.1	TCP	66	80 → 58300 [ACK] Seq=1 Ack=74 Win=65536 Len=0 TSval=386733828

5. Source and Destination Port Analysis

The connection used 127.0.0.1 as both the source and destination IP address because the communication remained on the loopback interface. The client selected the temporary source port 58300, while the local HTTP server listened on destination port 80. The response traffic reversed these port roles from 80 to 58300.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.1	TCP	74	58300 → 80 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TS
2	0.000013172	127.0.0.1	127.0.0.1	TCP	74	80 → 58300 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 S
3	0.000022029	127.0.0.1	127.0.0.1	TCP	66	58300 → 80 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=3867338285
4	0.000059952	127.0.0.1	127.0.0.1	HTTP	139	GET / HTTP/1.1
5	0.000063639	127.0.0.1	127.0.0.1	TCP	66	80 → 58300 [ACK] Seq=1 Ack=74 Win=65536 Len=0 TSval=386733828
6	0.001313448	127.0.0.1	127.0.0.1	TCP	223	80 → 58300 [PSH, ACK] Seq=1 Ack=74 Win=65536 Len=157 TSval=38
7	0.001324871	127.0.0.1	127.0.0.1	TCP	66	58300 → 80 [ACK] Seq=74 Ack=158 Win=65536 Len=0 TSval=3867338
8	0.001340270	127.0.0.1	127.0.0.1	HTTP	2953	HTTP/1.0 200 OK (text/html)
9	0.001343362	127.0.0.1	127.0.0.1	TCP	66	58300 → 80 [ACK] Seq=74 Ack=3045 Win=95744 Len=0 TSval=386733
10	0.001364543	127.0.0.1	127.0.0.1	TCP	66	80 → 58300 [FIN, ACK] Seq=3045 Ack=74 Win=65536 Len=0 TSval=3
11	0.001458035	127.0.0.1	127.0.0.1	TCP	66	58300 → 80 [FIN, ACK] Seq=74 Ack=3046 Win=95744 Len=0 TSval=3
12	0.001472016	127.0.0.1	127.0.0.1	TCP	66	80 → 58300 [ACK] Seq=3046 Ack=75 Win=65536 Len=0 TSval=386733

6. TCP Connection Teardown

Connection termination packets were visible in the capture. The server sent a FIN-ACK from port 80 to client port 58300, the client replied with a FIN-ACK from 58300 to port 80, and the server completed the teardown with a final ACK. This showed the orderly termination of the local TCP session.

3	0.000022029	127.0.0.1	127.0.0.1	TCP	66	58300 → 80 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=3867338285
4	0.000059952	127.0.0.1	127.0.0.1	HTTP	139	GET / HTTP/1.1
5	0.000063639	127.0.0.1	127.0.0.1	TCP	66	80 → 58300 [ACK] Seq=1 Ack=74 Win=65536 Len=0 TSval=386733828
6	0.001313448	127.0.0.1	127.0.0.1	TCP	223	80 → 58300 [PSH, ACK] Seq=1 Ack=74 Win=65536 Len=157 TSval=38
7	0.001324871	127.0.0.1	127.0.0.1	TCP	66	58300 → 80 [ACK] Seq=74 Ack=158 Win=65536 Len=0 TSval=3867338
8	0.001340270	127.0.0.1	127.0.0.1	HTTP	2953	HTTP/1.0 200 OK (text/html)
9	0.001343362	127.0.0.1	127.0.0.1	TCP	66	58300 → 80 [ACK] Seq=74 Ack=3045 Win=95744 Len=0 TSval=386733
10	0.001364543	127.0.0.1	127.0.0.1	TCP	66	80 → 58300 [FIN, ACK] Seq=3045 Ack=74 Win=65536 Len=0 TSval=3
11	0.001458035	127.0.0.1	127.0.0.1	TCP	66	58300 → 80 [FIN, ACK] Seq=74 Ack=3046 Win=95744 Len=0 TSval=3
12	0.001472016	127.0.0.1	127.0.0.1	TCP	66	80 → 58300 [ACK] Seq=3046 Ack=75 Win=65536 Len=0 TSval=386733

7. Sequence of Events Review

The captured session followed the expected TCP lifecycle. The client first initiated the connection, the server acknowledged and accepted it, and the client completed the handshake. HTTP application data was then exchanged before both endpoints closed the connection using FIN and ACK flags.

Step	Packet	Source	Destination	Behavior
1	SYN	127.0.0.1:58300	127.0.0.1:80	Connection request
2	SYN, ACK	127.0.0.1:80	127.0.0.1:58300	Server acknowledgment
3	ACK	127.0.0.1:58300	127.0.0.1:80	Handshake completed
4	HTTP GET	127.0.0.1:58300	127.0.0.1:80	Application request
5	HTTP 200 OK	127.0.0.1:80	127.0.0.1:58300	Application response
6	FIN, ACK	127.0.0.1:80	127.0.0.1:58300	Server begins teardown
7	FIN, ACK	127.0.0.1:58300	127.0.0.1:80	Client closes its side
8	ACK	127.0.0.1:80	127.0.0.1:58300	Connection closed

8. Conclusions

The TCP analysis was successfully completed using an authorized local HTTP service on Kali Linux. Wireshark captured the complete connection lifecycle, including the SYN, SYN-ACK, and ACK packets used to establish the session, the client and server port roles, and the FIN-ACK and ACK packets used during connection teardown. The results confirmed how TCP establishes, maintains, and orderly terminates a connection within a controlled local laboratory environment.